

MACHINE LEARNING

pattern
→ insight
→ tác động

CƠ BẢN

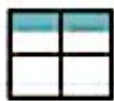
NỀN TẢNG VỮNG CHẮC BƯỚC VÀO AI

— GHI CHÚ ĐẦY ĐỦ, CAO CẤP —
Supervised • Metrics • Models • Pipeline

Từ bảng dữ liệu & biểu đồ đến các model có khả năng generalize —
hành trang vững chắc bước vào AI.



ML học pattern từ data để đưa ra prediction — không phải phép màu, chỉ là **fit + evaluate + iterate**.



Data



x1	x2	y

Features /
Labels



Train Model



Evaluate



Iterate



Hướng tới AI

SUPERVISED • OVERFIT • REGRESSION • CLASSIFY • METRICS • CV • NN TEASER

BÊN TRONG CÓ GÌ

- ✓ các loại bài toán ML
- ✓ pipeline end-to-end
- ✓ bias-variance trade-off
- ✓ model linear & logistic
- ✓ trees & ensembles
- ✓ clustering (cơ bản)
- ✓ feature engineering
- ✓ cross-validation

BẢNG SỐ LIỆU

- ✓ 20+ trang ghi chú viết tay
- ✓ diagram & sketch rõ ràng
- ✓ công thức tối giản, nặng về trực giác
- ✓ các sai lầm thường gặp

LỘ TRÌNH CHỦ ĐỀ

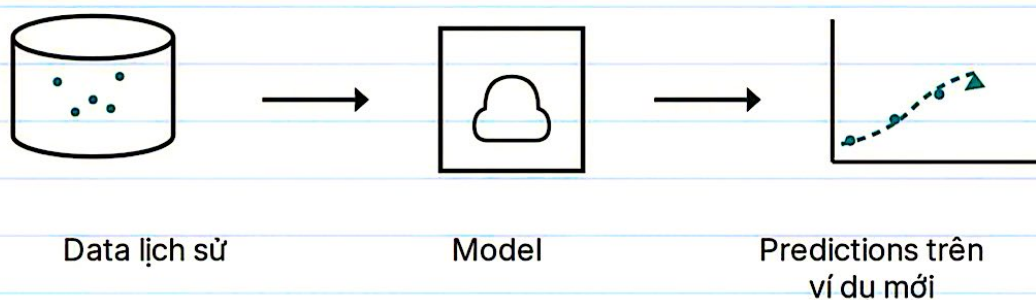
- ✓ data → hiểu
- ✓ feature → biểu diễn
- ✓ model → học
- ✓ evaluate → đo lường
- ✓ iterate → cải thiện

chặng tiếp theo:

01 — ML LÀ GÌ & TẠI SAO CẦN ML

- ① **ĐỊNH NGHĨA**: Machine Learning = các algorithm tự cải thiện khi thực hiện một task bằng kinh nghiệm (data) thay vì chỉ dựa vào rule viết sẵn.
- ② **GHI CHÚ CẦN NỐI**: Data Analysis đi tìm insight; ML biến pattern thành predictor. Series AI (RAG/agents) sẽ xây trên nền tảng này.

③ BỨC TRANH TỔNG THỂ:



- ④ **KHI NÀO ML HỮU ÍCH**: ranking, classification, forecasting, recommendation – khi pattern có thể học được và có data.
- ⑤ **KHI NÀO KHÔNG**: data quá ít, cần rule nhân quả chính xác, hoặc bài toán thuần kỹ thuật.
- ⑥ **PHÒNG VẤN**: ML vs lập trình cổ điển –

ML
rule từ data
(được học)

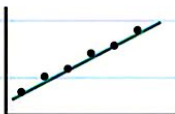
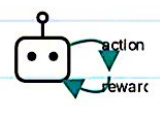
Lập trình cổ điển
rule từ code
(tưởng minh)

- ⑦ **SAI LẦM**: train trên toàn bộ data rồi tuyên bố thành công – không có tập test riêng.

↪ **TÓM LẠI**: ML được đo bằng khả năng generalize –

02 — CÁC LOẠI ML

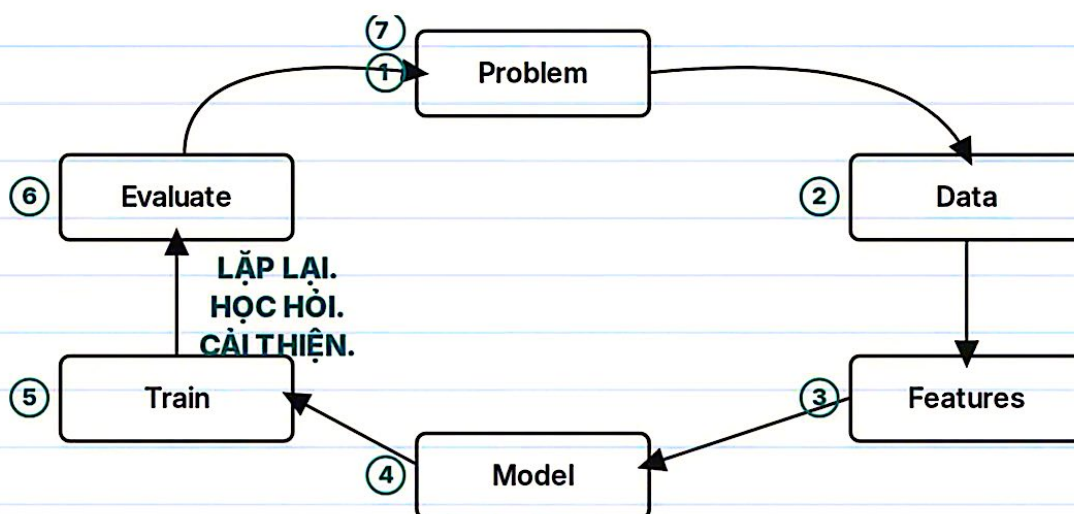
- ① **Supervised Learning**: học từ data có nhãn ($X \rightarrow y$).
 - Regression: dự đoán số (v.d., giá nhà 3.45 tỷ)
 - Classification: dự đoán nhóm/lớp (v.d., spam / không spam)
- ② **Unsupervised Learning**: tìm cấu trúc trong data **không có nhãn**.
 - Clustering: nhóm các data giống nhau (v.d., khách hàng)
 - Dimensionality Reduction: nén thông tin (v.d., PCA, t-SNE)
- ③ **Reinforcement Learning**: agent học bằng cách thực hiện hành động và nhận **reward theo thời gian**.
 - Thử, nhận feedback, cải thiện, lặp lại.
 - (gợi mở – sẽ nói kỹ hơn ở phần AI/agents sau!)

SUPERVISED	UNSUPERVISED	REINFORCEMENT
		
input $X \rightarrow$ output y labeled data 🏷️	input $X \rightarrow$ structure no labels 🚫	agent học qua reward theo thời gian

- ④ **Semi-supervised / Self-supervised**: lai giữa hai loại hoặc dùng pretext không nhãn
 - Deep learning hiện đại dùng nhiều self-sup (v.d., contrastive, masked modeling) **trước khi** fine-tune trên ít data có nhãn.
- ⑤ **Mẹo phòng vấn**: Chọn supervised khi bạn có **target rõ ràng (y)** cần dự đoán từ input (X).
- ⑥ **Sai lầm thường gặp**: Gọi mọi thứ là "AI" 😊 mà không nói rõ

03 — PIPELINE ML ★

1) PIPELINE (VÒNG LẶP LẶP LẠI)



2) TRAIN vs SERVE

TRAINING

Học parameter (θ) từ data.

SERVING (INFERENCE)

Dùng model đã train (θ cố định) để đưa ra prediction trên data mới.

3) TỪ DATA ANALYSIS: EDA & làm sạch data vẫn phải làm trước – **GARBAGE IN, GARBAGE OUT.**

4) BASELINE TRƯỚC: model đơn giản trước khi phức tạp. ★

5) PHÒNG VẤN : đi qua pipeline cho mọi câu hỏi về ML project.

6) SAI LẦM THƯỜNG GẶP : ×

- Nhảy thẳng vào neural net trước khi có baseline và tách split sạch.

★ **TÓM LẠI:** Kỷ luật với pipeline quan trọng hơn

04 — FEATURES, LABELS & SPLITS

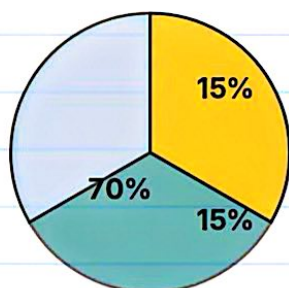
① Features (X) = input; Label (y) = cái bạn dự đoán (supervised).

② Ví dụ một dòng: house size, số phòng → price.

house_size (sqft)	rooms	price (\$)
1200	3	250,000

③ Train / Validation / Test: học / tinh chỉnh / báo cáo cuối – test **chỉ một lần** là lý tưởng.

④ Tỉ lệ split điển hình (ví dụ):

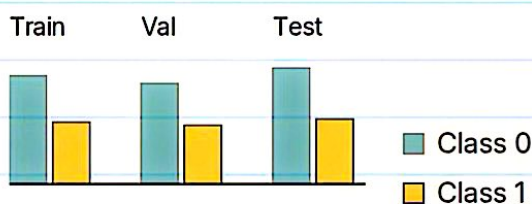


■ Train (70%)
 ■ Validation (15%)
 ■ Test (15%)

v.d. 70/15/15

tỉ lệ này thay đổi tùy trường hợp!

⑤ Stratify split cho classification để giữ **class balance**.



⑥ **Phong vấn:** tại sao cần test set?

→ ước lượng hiệu năng **thực tế (real-world)** mà không gian lận.

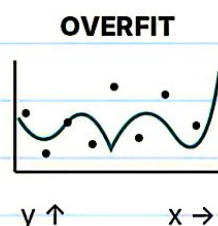
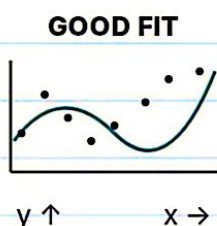
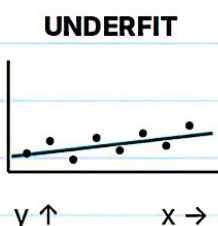
✗ Sai lầm: nhìn trước test khi đang tuning = **nói dối lạc quan.**

05 — OVERFIT & UNDERFIT

① **UNDERFIT**: model quá đơn giản – lỗi cao CẢ train và test (bỏ lỡ pattern).

② **OVERFIT**: học thuộc train – lỗi train thấp, lỗi test cao.

③ **SKETCH MINH HOẠ**:



④ **BIAS-VARIANCE INTUITION (NHẸ NHÀNG)**:

High bias $\approx$ underfit (model quá đơn giản để học pattern)

High variance $\approx$ overfit (model quá nhạy với noise)

⑤ **CÁCH KHẮC PHỤC**:

- Nhiều data hơn 📁
- Model đơn giản hơn 📦
- Regularization (L2, Dropout, v.v.) λ
- Early stopping 🕒
- Feature tốt hơn 💡

⑥ **PHÒNG VẤN**: chẩn đoán bằng cách so sánh train vs validation metric.

⚠️ **SAI LẦM**: chỉ theo dõi training accuracy.

06 — LINEAR REGRESSION

- ① Dự đoán y liên tục dạng tổng có trọng số của feature + bias:

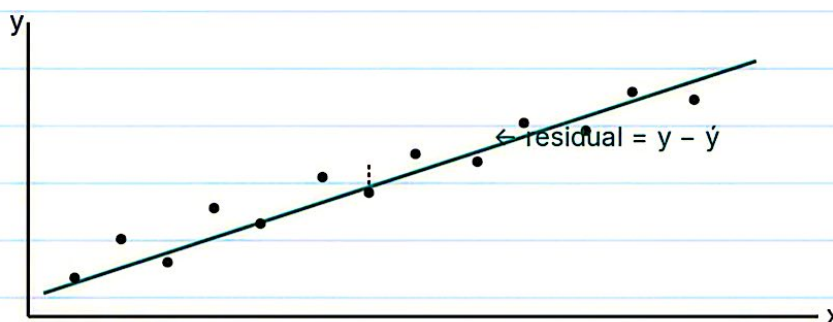
$$\hat{y} = w \cdot x + b$$

w : weights

x : features

b : bias

- ② Sketch đường fit nhỏ qua các điểm scatter; residual = $y - \hat{y}$.



- ③ Loss idea: **MSE** – trung bình bình phương lỗi; minimize để học w, b .

$$\text{MSE} = 1/n \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

càng thấp càng tốt

- ④ Diễn giải weight (hướng/độ mạnh) cẩn thận – **correlation $\neq$ causation**.

- ⑤ Assumption nhẹ nhàng: quan hệ roughly linear giúp ích; thường phải scale feature.

- ⑥ **Phỏng vấn**: closed-form vs gradient descent khi học weight.

- Closed-form (Normal Equation): chính xác, nhanh cho data nhỏ/vừa.
- Gradient Descent: scale tốt cho data lớn, hoạt động tốt với regularization, online update.

Closed-form:

$$w = (X^T X)^{-1} X^T y$$

Gradient Descent:

$$w \leftarrow w - \eta \nabla_w J(w)$$

- ⚠ Sai lầm: dùng **accuracy** cho regression – dùng **MAE / MSE / R^2** .

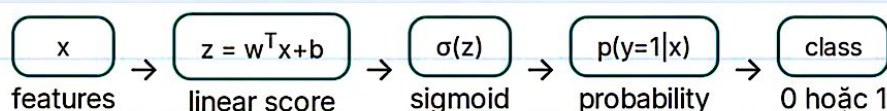
MAE: mean |error|

MSE: mean squared error

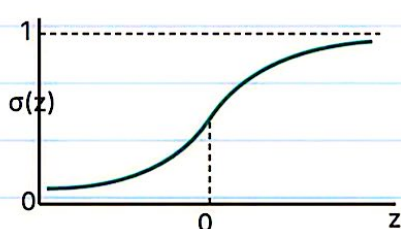
R^2 : variance explained

07 — CLASSIFICATION & LOGISTIC

- ① Classification: dự đoán lớp rời rạc (spam/ham, fraud/ok).
- ② Logistic regression: linear score $\rightarrow$ sigmoid $\rightarrow$ probability; threshold để ra lớp.



- ③ Sigmoid nén score thành probability (0 đến 1).



Decision boundary:

tại threshold **0.5**

(có thể thay đổi)

$p \geq 0.5 \rightarrow \text{class 1}$

$p < 0.5 \rightarrow \text{class 0}$

- ④ Softmax (gợi mở): cho multi-class (K lớp).

$$z_k \rightarrow \exp(z_k) / \sum_j \exp(z_j) = p(y=k|x) \quad (\sum_k p_k = 1)$$

- ⑤ Tại sao không dùng linear regression thường cho class?
 - Output không giới hạn trong **[0,1]** $\rightarrow$ không phải probability hợp lệ.
 - Không model tốt được quan hệ dạng chữ S.

- ⑥ **Phỏng vấn:** probability calibration vs hard label.

- Calibration = predicted prob khớp với tần suất thực tế.
- v.d., trong các case predict 0.8–0.9, **~80–90%** là positive.

- ⑦ Sai lầm: coi probability **0.9** là chắc chắn tuyệt đối mà không check metric trên data imbalance.

Probability cao $\neq$ precision cao trên data imbalanced.

08 — CÁC METRIC QUAN TRỌNG

① Confusion matrix (binary classification)

	Predicted	
	(+)	(-)
Actual (+)	TP (true positive)	FN (false negative)
(-)	FP (false positive)	TN (true negative)

Ghi chú:

- Actual + : thực sự positive
- Actual - : thực sự negative
- Predicted + : model nói positive
- Predicted - : model nói negative

② Accuracy có thể nói dối trên data **imbalanced**.

- Ví dụ: accuracy 99% bằng cách predict toàn negative khi positive hiếm.

③ Precision, Recall, F1

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \quad \text{Recall} = \text{TP} / (\text{TP} + \text{FN}) \quad \text{F1} = 2 \times \text{P} \cdot \text{R} / (\text{P} + \text{R})$$

- Precision: trong các dự đoán positive, bao nhiêu cái đúng?
- Recall (Sensitivity): trong các positive thực tế, bắt được bao nhiêu?
- F1: harmonic mean – cân bằng precision & recall.

④ ROC-AUC (Receiver Operating Characteristic – Area Under Curve)

- Vẽ TPR (Recall) theo FPR (1-Specificity) qua các threshold.
- AUC đo **chất lượng xếp hạng** (0.5 = ngẫu nhiên, 1.0 = hoàn hảo).

⑤ Metric cho Regression

- MAE (Mean Absolute Error) = trung bình $|y - \hat{y}|$ → ít bị outlier ảnh hưởng
- MSE (Mean Squared Error) = trung bình $(y - \hat{y})^2$ → phạt nặng lỗi lớn
- RMSE = $\sqrt{\text{MSE}}$ → cùng đơn vị với target
- R^2 (Coefficient of Determination) = $1 - \text{SSE}/\text{SST}$ → % variance giải thích được (1 là tốt nhất)

⑥ Interview insight: chọn metric dựa trên chi phí FP vs FN.

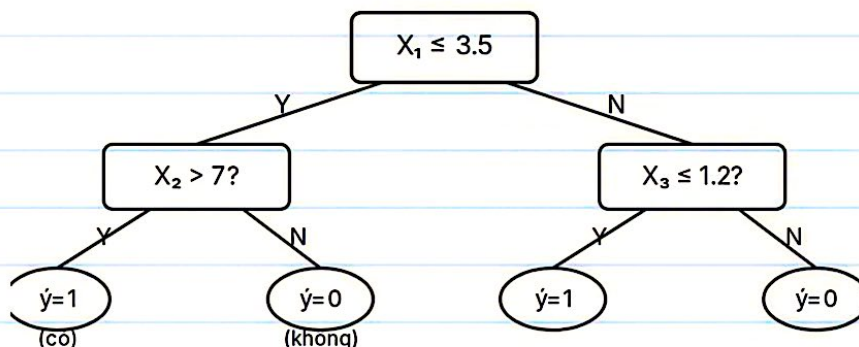
- Chẩn đoán y tế → bỏ sót bệnh (FN) tốn kém → tối ưu Recall.
- Phát hiện spam → chặn nhầm mail thật (FP) tốn kém → tối ưu Precision.

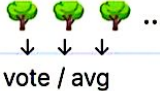
⑦ Sai lầm thường gặp

- Chỉ tối ưu **accuracy** cho sự kiện hiếm.
- Nó che giấu hiệu năng kém trên lớp thiểu số.

09 — TREES & FORESTS

- ① Decision tree: đặt câu hỏi trên feature → leaf prediction; interpretable.
- ② Sketch tree nhỏ: split trên feature.

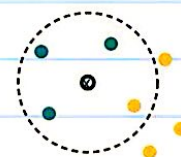


- ③ Overfit risk: tree sâu sẽ học thuộc – prune / giới hạn depth.
- ④ Random Forest: nhiều tree trên bootstrap sample + feature ngẫu nhiên → average / vote. 
- ⑤ Strengths: decision boundary phi tuyến, xử lý được mixed feature type; chú ý leakage & imbalance.
- ⑥ **Phòng vấn**: bagging giảm variance; một tree đơn có variance cao.
- ⑦ **Sai lầm**: đọc feature importance như causation thuần túy.

★ **TÓM LẠI**: Một tree giải thích được,

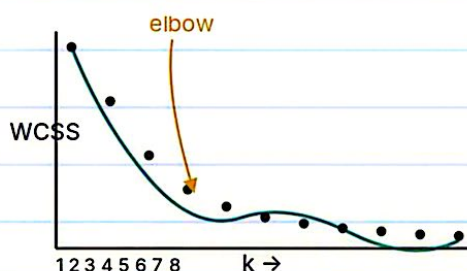
10 — k-NN & CLUSTERING

- ① k-NN: classify/regress bằng majority/average của k nearest neighbor – lazy learner, khoảng cách quan trọng (scale feature!).



● class A ● class B x new point — k = 5 → predict A (3 vs 2)

- ② k-means: unsupervised – chọn k centroid, gán điểm, cập nhật tâm, lặp lại đến khi (gần) hội tụ.
- ③ Chọn k (ý tưởng elbow): vẽ WCSS (within-cluster sum of squares) theo k. Tìm điểm "elbow" – nơi mức cải thiện bắt đầu chững lại.



- ④ Khi nào dùng:
- baseline đơn giản
 - nhóm/segment mang tính khám phá
 - cẩn thận: curse of dimensionality → khoảng cách kém ý nghĩa hơn khi số chiều tăng.
- ⑤ Góc nhìn phòng vấn:
- k-NN = supervised, vote/average theo neighbor
 - k-means = unsupervised, khám phá cluster
- ⑥ Sai lầm thường gặp:
- dùng k-means cho shape không phải blob mà không suy nghĩ (v.d., moons, concentric circles).
 - quên scale feature cho k-NN (hoặc k-means).

11 — FEATURE ENGINEERING ✨

- ① Feature **tạo nên** hoặc **phá hỏng** model – domain knowledge + transform.
- ② Các thao tác phổ biến: scaling (standardize/normalize), one-hot categorical, binning, log transform, interaction.
- ③ **Missing value**: impute cẩn thận; việc thiếu data cũng có thể là một signal.
- ④ Before/after nhỏ: raw column → ma trận sẵn sàng cho model.

RAW COLUMNS					→	MA TRẬN SẴN SÀNG CHO MODEL					
user_id	age	country	income	signup_date		age_z	income_log	US	IN	FR	month
1	23	US	55000	2023-01-15		-1.02	10.91	1	0	0	1
2	35	IN	—	2023-02-20		0.45	10.40	0	1	0	2
3	?	FR	72000	2023-03-05		0.00	10.79	0	0	1	3
4	28	US	48000	2023-04-11		-0.60	10.78	1	0	0	4
5	31	IN	61000	2023-05-22		-0.12	10.92	0	1	0	5

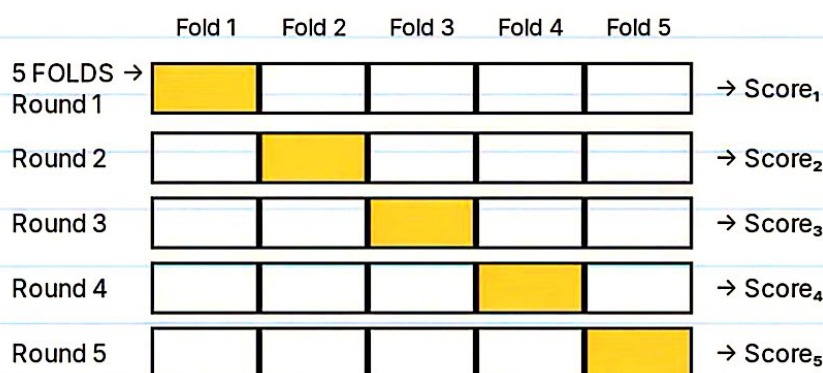
(age đã impute, income log, country one-hot, month được trích xuất)

- ⑤ Từ Data Analysis: EDA giúp định hướng feature nào cần tạo.
- ⑥ **Phòng vãn**: feature bị **leaky** (dùng thông tin tương lai) sẽ phá hỏng tính hợp lệ.
- ⑦ **Sai lầm**: scale bằng toàn bộ dataset kể cả test – chỉ nên fit scaler trên train.

★ **TÓM LẠI: Feature tốt thường đánh bại model phức tạp** ★

12 — CROSS-VALIDATION

- ① k-fold CV: chia train thành k fold; xoay vòng validation; average score – ổn định hơn so với 1 split.
- ② Sơ đồ nhỏ: 5 fold, mỗi vòng giữ lại 1 fold.



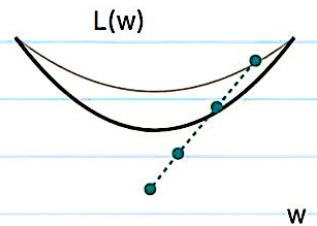
} average score

- ③ Dùng CV để chọn model / hyperparameter; giữ lại một final test set riêng.
- ④ Time series: dùng split time-aware – không random shuffle làm lộ tương lai.
- ⑤ Nhận biết Nested CV (heavy) – biết nó tồn tại để tuning trung thực.
- ⑥ **Phòng vấn:** variance của một validation score đơn lẻ → CV giúp ích.
- ⑦ **Sai lầm:** CV trên time series đã shuffle; hoặc tuning trên test set.

★ **TÓM LẠI:** Luân phiên phần holdout,

13 — GRADIENT DESCENT

- 1) **Mục tiêu**: minimize loss $L(w)$ bằng cách đi xuống dốc trong không gian parameter.



learning rate η = step size

2) Step size quan trọng (η):

- η quá lớn $\rightarrow$ overshoot, này qua này lại, **diverge**.
- η quá nhỏ $\rightarrow$ step quá nhỏ, **rất chậm**.
- **Batch vs SGD vs Mini-batch**:
 - Batch GD: ổn định, gradient chính xác, tốn kém mỗi step.
 - SGD: **noisy update**, rẻ mỗi step, có thể zig-zag.
 - Mini-batch: **điểm cân bằng lý tưởng** trong thực tế.

3) Epochs = số lượt đi qua toàn bộ dataset.

4) Tại sao quan trọng: ý tưởng đơn giản nhưng scale lên được model khổng lồ (cầu nối vào deep learning).

5) Interview nugget:

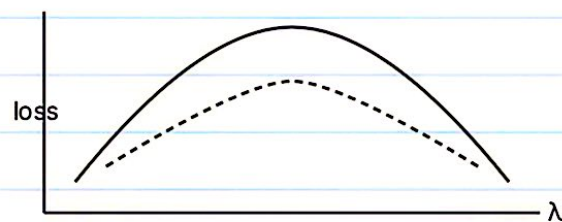
Gradient = hướng tăng nhanh nhất (ascent) – ta đi theo hướng ngược lại (descent).

6) Sai lầm thường gặp:

Quên scale feature $\rightarrow$ bài toán bị **ill-conditioned** (scale lệch) $\rightarrow$ **train chậm**.

14 — REGULARIZATION

- ① Regularization = phạt (penalty) lên độ phức tạp để chống **overfitting**.
- ② L2 (Ridge): thu nhỏ weight một cách mượt; L1 (Lasso): có thể đưa weight về 0 → sparsity / feature select.
- ③ Sketch nhỏ: $\text{loss} + \lambda \cdot \text{penalty}$



λ kiểm soát
độ mạnh.

- ④ Early stopping như một **regularization** ngầm.
- ⑤ **Dropout** – gợi mở cho neural net (sẽ nói sau).
- ⑥ **Phòng vấn:** bias-variance – regularization thường $\uparrow$ bias $\downarrow$ variance.

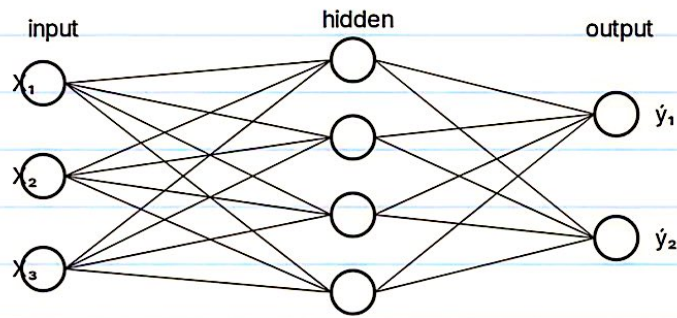
⚠ Sai lầm:

- λ quá lớn → underfit.
- $\lambda = 0$ với model khổng lồ → overfit.

15 — GỢI MỞ NEURAL NETS ★

- ① **Cầu nối vào AI**: neural net xếp chồng nhiều layer gồm weighted sum + nonlinear activation.

- ② **Sketch nhỏ:**



deep = nhiều layer.

- ③ **Cùng ý tưởng**: loss, gradient descent, train/val/test, overfit.
- ④ **Vì sao deep learning bùng nổ**: data + GPU + training tốt hơn – là nền tảng cho AI hiện đại (LLM/RAG sau này).
- ⑤ Bắt đầu với classical ML trước – rồi mới đến deep khi cần.
- ⑥ **Phòng vấn**: NN vẫn là function approximation + optimization.
- ⑦ **Sai lầm**: bỏ qua baseline và nhảy thẳng vào net khổng lồ.

16 — SAI LẦM & LEAKAGE ★

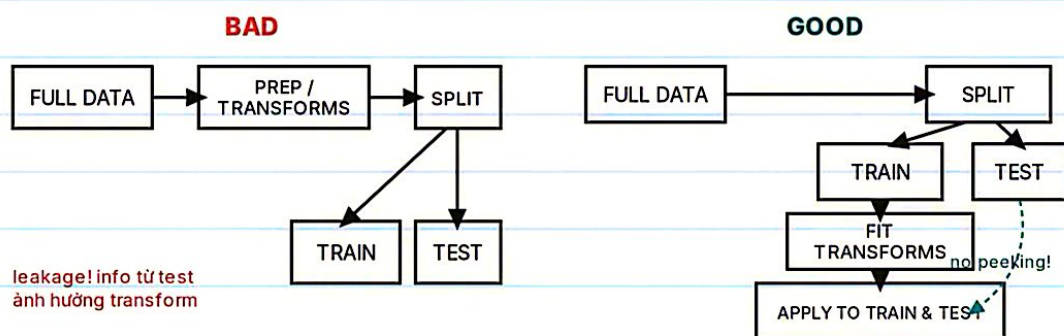
1) Data leakage: thông tin từ tương lai/test lọt vào training – score trông tuyệt vời, nhưng production fail.

2) Ví dụ:

- target encoding sai (dùng toàn bộ data)
- scaling với cả test
- random split trên time series
- dòng trùng lặp giữa các split

3) Class imbalance traps; phân phối thay đổi (drift).

4) Sketch pipeline **bad** vs **good** nhỏ:



5) Checklist:

- ☒ fit transform chỉ trên train
- ☒ split trước khi prep nặng
- ☒ khớp timing production (time window, lag, availability)

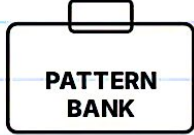
6) Phòng vấn: làm sao phát hiện leakage – metric quá tốt + feature đáng ngờ.

Sai lầm: ăn mừng 99% mà chưa audit leakage.

17 — MẪU CÂU HỎI PHÒNG VẤN ★

KHO MẪU CÂU HỎI

- 1 Supervised vs unsupervised
- 2 Bias-variance / chặn đoán overfit
- 3 Mục đích Train/val/test
- 4 Đánh đổi Precision vs recall
- 5 Giải thích linear/logistic regression
- 6 Vì sao random forest hơn 1 tree
- 7 Cross-validation khi nào/vì sao
- 8 Data leakage là gì
- 9 Trình bày một ML project từ đầu đến cuối



PATTERN
BANK

CÁCH TIẾP CẬN (dùng mỗi lần!)

1 ĐỊNH NGHĨA

→

2 SKETCH

→

3 CHỌN METRIC

→

4 FAILURE MODE

Làm rõ bài toán &
assumption.

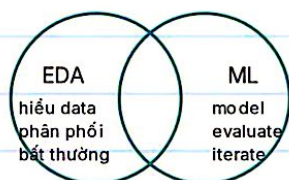


- Regression → RMSE/MAE
- Classification → F1/AUC
- Ranking → NDCG

Điều gì có thể sai?
Bạn phát hiện thế
nào?

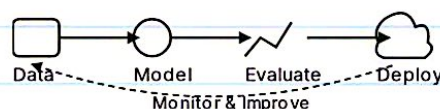


💡 MẸO: KẾT NỐI VỚI DATA ANALYSIS (EDA) VÀ AI STACK TIẾP THEO



EDA tốt
→ feature tốt hơn &
ít leak hơn!

AI STACK TIẾP THEO



! SAI LẦM THƯỜNG GẶP

❌ Liệt kê công thức mà không có use case.
Người phỏng vấn muốn nhận định, không phải trí
nhớ.

THAY VÀO ĐÓ HÃY LÀM

✅ Kể một câu chuyện. Dùng pattern.
Nêu rõ trade-off. Gắn với mục tiêu business.

18 — CHEAT SHEET

1) CÁC LOẠI ML

- SUPERVISED → data có nhãn (x, y)
- UNSUPERVISED → data không nhãn (x)
- REINFORCEMENT LEARNING → agent học bằng reward/penalty

2) PIPELINE ML (TỔNG QUAN)



lặp lại → theo dõi thí nghiệm → giám sát khi production

3) SPLIT + CROSS-VALIDATION

- TRAIN / VAL / TEST (v.d., 70/15/15)
- TEST = kiểm tra cuối. Không đụng khi dev.
- K-FOLD CV ($k=5$ hoặc 10 phổ biến)
 - chia thành k fold
 - train trên $k-1$, validate trên 1
 - lặp k lần, lấy trung bình score

4) OVERFITTING: DẤU HIỆU & CÁCH SỬA

DẤU HIỆU: train score ↑↑ nhưng val/test score ↓

khoảng cách lớn giữa train và val
variance cao, CV score bất ổn

CÁCH SỬA: • nhiều data hơn • model đơn giản hơn

- data augmentation • feature selection
- regularization (L1/L2) • cross-validation
- dropout / early stopping • ensembling

5) LINEAR & LOGISTIC REGRESSION

LINEAR: $\hat{y} = w \cdot x + b$ (cho regression)

LOGISTIC: $\text{logit}(z) = w \cdot x + b$, $p = \sigma(z) = 1/(1 + e^{-z})$
(cho binary classification)

Predict 1 nếu $p \geq 0.5$, ngược lại 0

6) CÁC METRIC ĐÁNH GIÁ

CLASSIFICATION (binary):

Metric	Formula
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$
Precision	$TP/(TP+FP)$
Recall	$TP/(TP+FN)$
F1	$2 \cdot P \cdot R / (P+R)$

REGRESSION:

MAE = $1/n \sum |y_i - \hat{y}_i|$ (càng thấp càng tốt)

MSE = $1/n \sum (y_i - \hat{y}_i)^2$ (càng thấp càng tốt)

7) CÁC ALGORITHM PHỔ BIẾN (TỔNG QUAN)

- TREES: dễ diễn giải, xử lý phi tuyến, có feature importance
- RANDOM FOREST: bagging nhiều tree, mạnh mẽ, giảm variance
- k-NN: lazy learner, dựa trên khoảng cách, cần scale feature!
- k-MEANS: clustering, minimize within-cluster variance, chọn k (elbow/silhouette)

8) FEATURE & PREPROCESSING

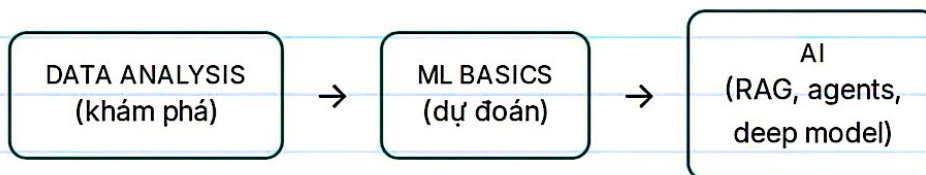
- SCALE FEATURE SỐ (StandardScaler / MinMaxScaler)
- ENCODE FEATURE CATEGORICAL (One-Hot / Ordinal / Target)
- ★ FIT TRANSFORM TRÊN TRAIN ONLY, RỒI TRANSFORM VAL/TEST!

9) OPTIMIZATION & REGULARIZATION

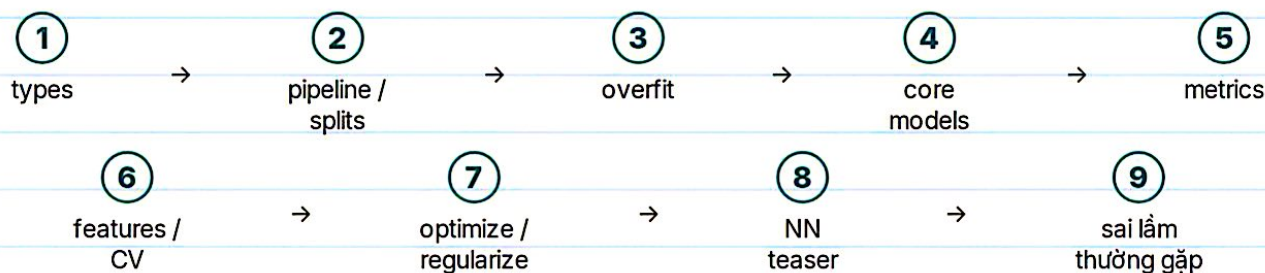
- GRADIENT DESCENT UPDATE: $w := w - \eta \nabla_w J(w)$ (η =learning rate, J =loss)
- η QUÁ LỚN → overshoot/diverge; η QUÁ NHỎ → hội tụ chậm
- REGULARIZATION: L1 (LASSO): $\lambda \|w\|_1$ → sparse; L2 (RIDGE): $\lambda \|w\|_2^2$ → thu nhỏ weight

19 — LỘ TRÌNH & TỔNG KẾT

1) Cầu nối series:



2) Hành trình trong cuốn sách này:

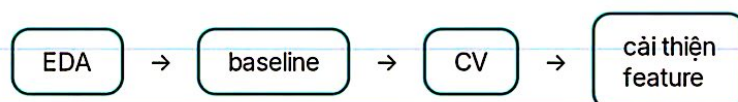


3) Điều quan trọng nhất:

ML biến data thành predictor đã được kiểm chứng – giữ lại phần sự thật (holdout), chọn đúng metric, đừng tin vào phép màu.

4) Vòng lặp luyện tập:

một bài toán tabular kiểu Kaggle



↶ (quay lại EDA và lặp)

5) Tiếp theo (sắp ra mắt):

deep learning, NLP/LLM, ghi chú RAG, hệ thống agentic – sau khi classical ML đã vững.

6) Lời kết:

Từ bảng dữ liệu đến trí tuệ,